

AER1416: Numerical Methods for Uncertainty Quantification — Assignment 2

Zouhair Adam Hamaimou Student #1004891986

Introduction and Problem Statement

This report estimates the mean $\mu = \mathbb{E}[\phi]$ and variance $\sigma^2 = \text{Var}[\phi]$ of three benchmark problems (P1–P3) using Latin Hypercube Sampling (Q1), Sobol quasi-Monte Carlo (Q2), and sparse Clenshaw–Curtis quadrature (Q3). All random inputs are uniformly distributed as required by the assignment. Questions Q4–Q6 cover importance sampling, Multilevel Monte Carlo [1], and O’Hagan’s critique [2].

P1 — 10-D quadratic ($x_i \sim \mathcal{U}[-1, 1]$).

$$f(\mathbf{x}) = a_0 + \sum_{i=1}^d a_i x_i + \gamma \sum_{i=1}^d \sum_{j=1}^d a_{ij} x_i x_j, \quad d = 10, \quad a_0 = 1, \quad a_i = \frac{1}{\sqrt{i}},$$

$$a_{ij} = \begin{cases} 1/i & i = j, \\ \min(i, j) / [\max(i, j)^2 (i + j)] & i \neq j, \end{cases}$$

where γ is a prescribed scaling factor.

P2 — Mass–spring–damper (solved with ode45).

$$m\ddot{u} + c\dot{u} + ku = F_0 \cos(\omega t), \quad u(0) = \dot{u}(0) = 0,$$

$$m \sim \mathcal{U}[0.9, 1.1], \quad c \sim \mathcal{U}[0.8, 1.2], \quad k \sim \mathcal{U}[10, 12], \quad \omega \sim \mathcal{U}[0.1, 0.2], \quad F_0 \sim \mathcal{U}[0.9, 1.1].$$

P3 — Stochastic diffusion ($d = 3$, $n = 143$, $\phi = u_1$).

$$\mathbf{A}(\boldsymbol{\theta})\mathbf{u}(\boldsymbol{\theta}) = \mathbf{b}, \quad \mathbf{A} = \mathbf{A}_0 + \sum_{i=1}^3 \theta_i \mathbf{A}_i, \quad \theta_i \sim \mathcal{U}[-3, +3] \text{ i.i.d.}$$

Q1 — Latin Hypercube Sampling**Methodology**

Latin Hypercube Sampling (LHS) is a stratified sampling strategy that generalises one-dimensional stratified sampling to higher dimensions. The range of each variable is divided into N bins of equal probability, and N samples are drawn such that no two points share a bin in any single dimension. As a result, every one-dimensional projection of the design contains exactly one point per stratum, which reduces estimator variance relative to simple random sampling.

Concretely, for N samples in d dimensions with $x_i \sim \mathcal{U}[a_i, b_i]$, the provided routine `lhsamp(d,N,lb,ub)` returns a $d \times N$ matrix $\mathbf{X}$ whose i -th row is constructed by:

- (i) drawing one uniform variate from each of the N sub-intervals $[\frac{k-1}{N}, \frac{k}{N}]$, $k = 1, \dots, N$;
- (ii) applying a random permutation to that row independently of all other rows;
- (iii) rescaling from $[0, 1]$ to $[a_i, b_i]$.

The mean and variance of ϕ are estimated as

$$\hat{\mu} = \frac{1}{N} \sum_{n=1}^N \phi(\mathbf{x}^{(n)}), \quad \hat{\sigma}^2 = \frac{1}{N-1} \sum_{n=1}^N (\phi(\mathbf{x}^{(n)}) - \hat{\mu})^2.$$

Seven sample sizes are tested: $N \in \{10^3, 5 \times 10^3, 10^4, 2 \times 10^4, 5 \times 10^4, 10^5, 2 \times 10^5\}$.

Code Structure

`Q1.m` calls `lhsamp(d, N, lb, ub)` with problem-specific bounds ($[-1, 1]^{10}$ for P1, physical ranges for P2, $[-3, 3]^3$ for P3), so no additional rescaling is needed in the evaluation functions. P2 uses `ode45` with an anonymous ODE function; P3 caches `P3.mat` via a `persistent` variable to avoid repeated file reads. For the full-field P3 plots, the complete solution vector $\mathbf{u} \in \mathbb{R}^{143}$ is accumulated across all $N = 200\,000$ samples.

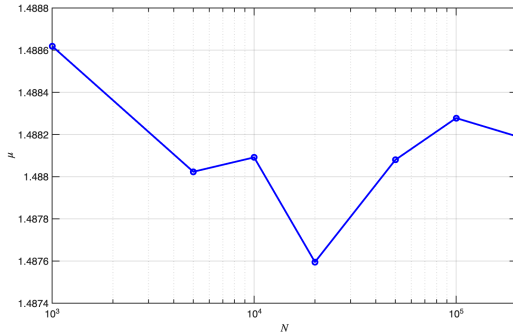
Results and Discussion

Table 1 reports $\hat{\mu}$ and $\hat{\sigma}^2$ for all sample sizes and problems.

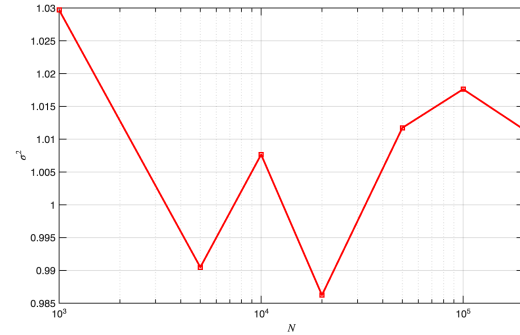
Table 1: LHS results for P1 ($\gamma = 0.5$, $x_i \sim \mathcal{U}[-1, 1]$), P2, and P3.

N	P1		P2		P3	
	$\hat{\mu}$	$\hat{\sigma}^2$	$\hat{\mu}$	$\hat{\sigma}^2$	$\hat{\mu}$	$\hat{\sigma}^2$
1 000	1.4878	1.0319	7.257×10^{-3}	6.521×10^{-4}	6.361×10^{-2}	7.257×10^{-5}
5 000	1.4879	1.0052	7.166×10^{-3}	6.492×10^{-4}	6.361×10^{-2}	7.220×10^{-5}
10 000	1.4881	0.9990	7.213×10^{-3}	6.505×10^{-4}	6.361×10^{-2}	7.216×10^{-5}
20 000	1.4884	1.0206	7.192×10^{-3}	6.509×10^{-4}	6.361×10^{-2}	7.224×10^{-5}
50 000	1.4882	1.0218	7.189×10^{-3}	6.496×10^{-4}	6.361×10^{-2}	7.219×10^{-5}
100 000	1.4881	1.0115	7.184×10^{-3}	6.498×10^{-4}	6.361×10^{-2}	7.219×10^{-5}
200 000	1.4883	1.0132	7.186×10^{-3}	6.494×10^{-4}	6.361×10^{-2}	7.219×10^{-5}

Table 1 and Figures 1–3 show the convergence of $\hat{\mu}$ and $\hat{\sigma}^2$ with N on a semi-log scale. For P1, the mean stabilises at $\hat{\mu} \approx 1.488$ and the variance at $\hat{\sigma}^2 \approx 1.013$, consistent from $N \geq 20\,000$ onward. For P2, the mean converges to approximately 7.19×10^{-3} and the variance to 6.49×10^{-4} by $N = 10\,000$. For P3, both moments are effectively converged even at $N = 1\,000$: $\hat{\mu} \approx 6.361 \times 10^{-2}$ and $\hat{\sigma}^2 \approx 7.22 \times 10^{-5}$. These results indicate that $N = 10\,000$ is a sufficient sample size for all three problems under LHS. The stratification property ensures tighter convergence than simple Monte Carlo at the same N , as each axis is covered uniformly by construction.

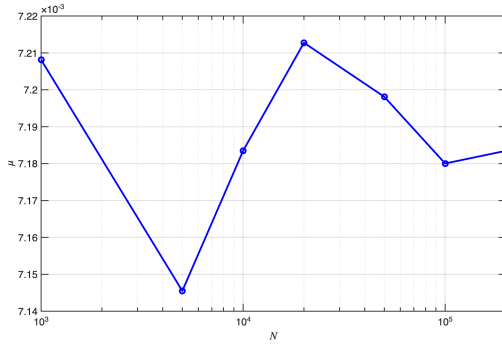


(a) Mean

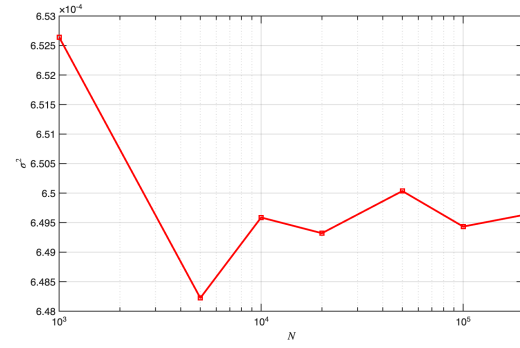


(b) Variance

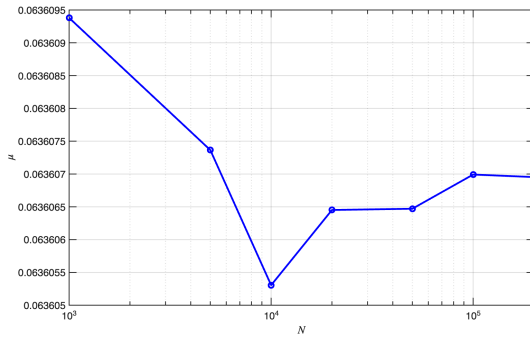
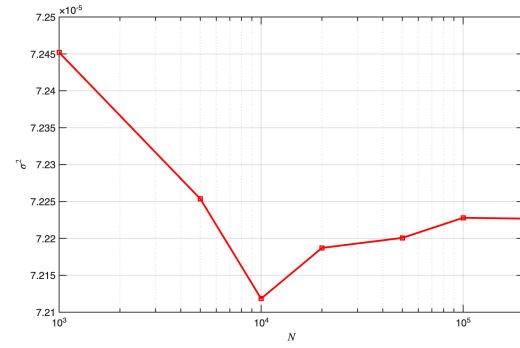
Figure 1: LHS convergence vs. N — P1.



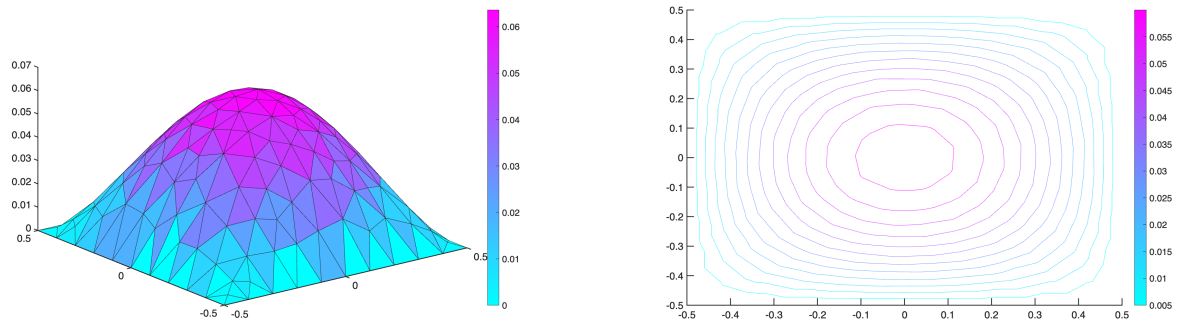
(a) Mean



(b) Variance

Figure 2: LHS convergence vs. N — P2.(a) Mean (u_1)(b) Variance (u_1)Figure 3: LHS convergence vs. N — P3.

Figures 4 and 5 show the full mean and variance fields over the P3 spatial mesh at $N = 200\,000$.

Figure 4: P3 mean field (LHS, $N = 200\,000$): 3-D surface (left) and contour (right).

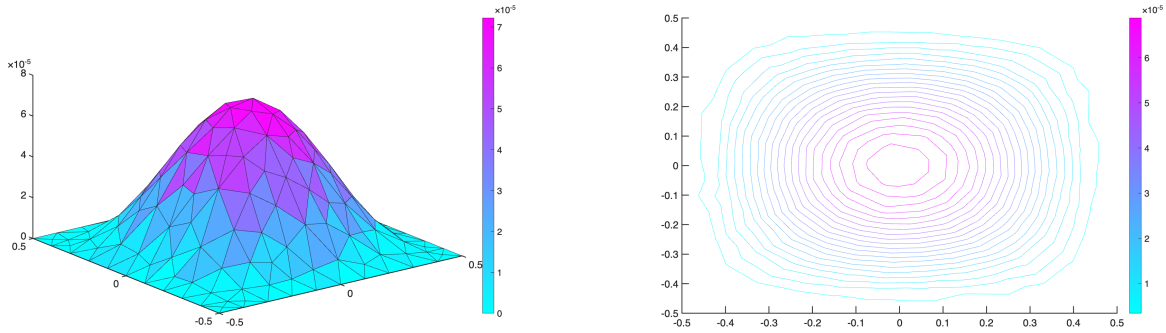


Figure 5: P3 variance field (LHS, $N = 200\,000$): 3-D surface (left) and contour (right).

Q2 — Quasi-Monte Carlo with Sobol Sequences

Methodology

Sobol sequences are low-discrepancy quasi-random point sets that fill the unit hypercube $[0, 1]^d$ far more uniformly than pseudo-random samples. Unlike LHS, which enforces stratification via random permutations, Sobol sequences are *deterministic*: successive points are placed to minimise gaps in coverage, giving faster convergence for smooth integrands.

The provided library `i4.sobol.generate(d, N, 0)` returns a $d \times N$ matrix of points in $[0, 1]^d$. Each column is then rescaled to the required parameter interval $[a_i, b_i]$ via

$$\theta_i = a_i + s_i (b_i - a_i),$$

where $s_i \in [0, 1]$ is the i -th Sobol coordinate. The mean and variance estimators are identical to those in Q1:

$$\hat{\mu} = \frac{1}{N} \sum_{n=1}^N \phi(\mathbf{x}^{(n)}), \quad \hat{\sigma}^2 = \frac{1}{N-1} \sum_{n=1}^N (\phi(\mathbf{x}^{(n)}) - \hat{\mu})^2.$$

Code Structure

`Q2.m` replaces `lhsamp` with `i4.sobol.generate(d, N, 0)`, which returns points in $[0, 1]^d$; these are rescaled to $[a_i, b_i]$ via a single vectorised operation `lb' + S .* (ub-lb)'`. The same evaluation functions and `var()` estimator from Q1 are reused. Since `skip=0` is used throughout, smaller designs are nested prefixes of larger ones, which contributes to the monotone convergence observed in Table 2.

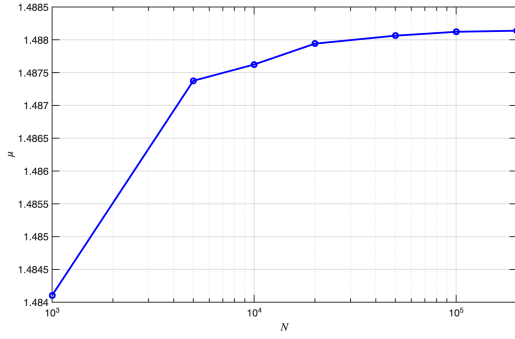
Results and Discussion

Table 2 reports the estimated moments across all sample sizes and problems. The Sobol estimates converge *monotonically* with N — in contrast to the small random fluctuations visible in the LHS curves — because the sequence deterministically fills gaps rather than relying on random permutations. All three problems stabilise by $N = 10\,000$, with P3 effectively converged already at $N = 1\,000$ due to its low dimensionality ($d = 3$). The converged values match the LHS and Assignment 1 MC benchmarks to at least three significant figures.

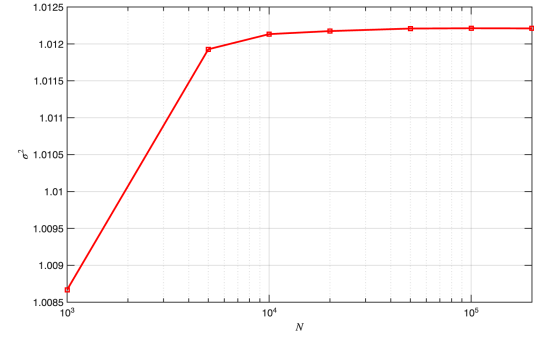
Figure ?? shows the semi-log convergence plots for all three problems, confirming the monotone behaviour described above.

Table 2: Sobol QMC results for P1 ($\gamma = 0.5$, $x_i \sim \mathcal{U}[-1, 1]$), P2, and P3.

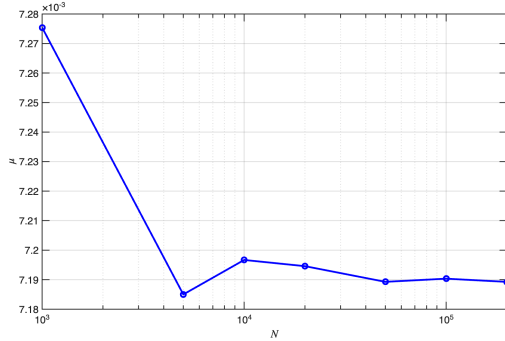
N	P1		P2		P3	
	$\hat{\mu}$	$\hat{\sigma}^2$	$\hat{\mu}$	$\hat{\sigma}^2$	$\hat{\mu}$	$\hat{\sigma}^2$
1 000	1.4841	1.0087	7.275×10^{-3}	6.496×10^{-4}	6.361×10^{-2}	7.232×10^{-5}
5 000	1.4874	1.0119	7.185×10^{-3}	6.498×10^{-4}	6.361×10^{-2}	7.224×10^{-5}
10 000	1.4876	1.0121	7.197×10^{-3}	6.496×10^{-4}	6.361×10^{-2}	7.223×10^{-5}
20 000	1.4879	1.0122	7.195×10^{-3}	6.496×10^{-4}	6.361×10^{-2}	7.222×10^{-5}
50 000	1.4881	1.0122	7.189×10^{-3}	6.496×10^{-4}	6.361×10^{-2}	7.221×10^{-5}
100 000	1.4881	1.0122	7.190×10^{-3}	6.496×10^{-4}	6.361×10^{-2}	7.221×10^{-5}
200 000	1.4881	1.0122	7.189×10^{-3}	6.496×10^{-4}	6.361×10^{-2}	7.221×10^{-5}



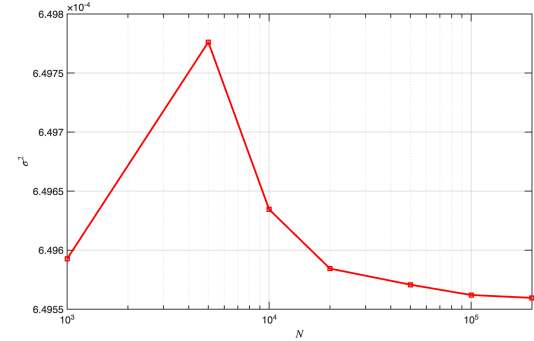
(a) Mean



(b) Variance

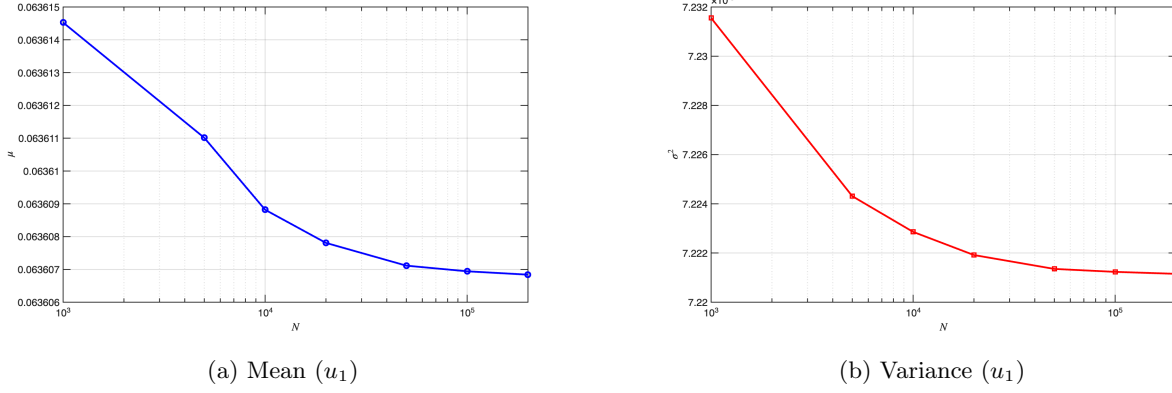
Figure 6: Sobol convergence vs. N — P1.

(a) Mean

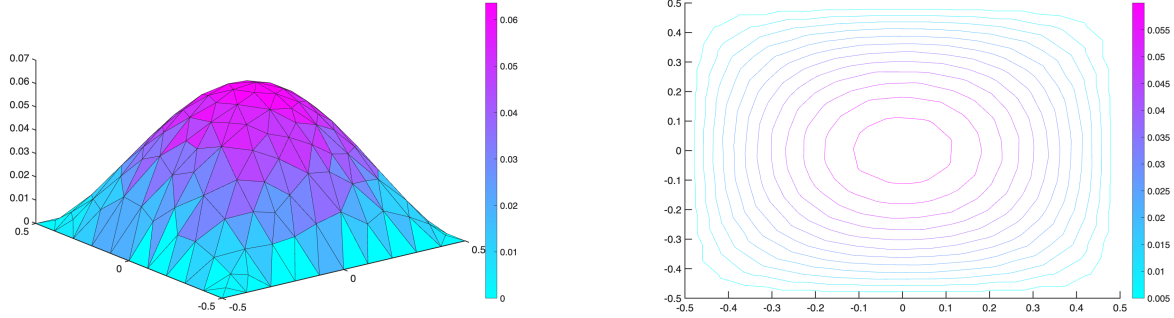
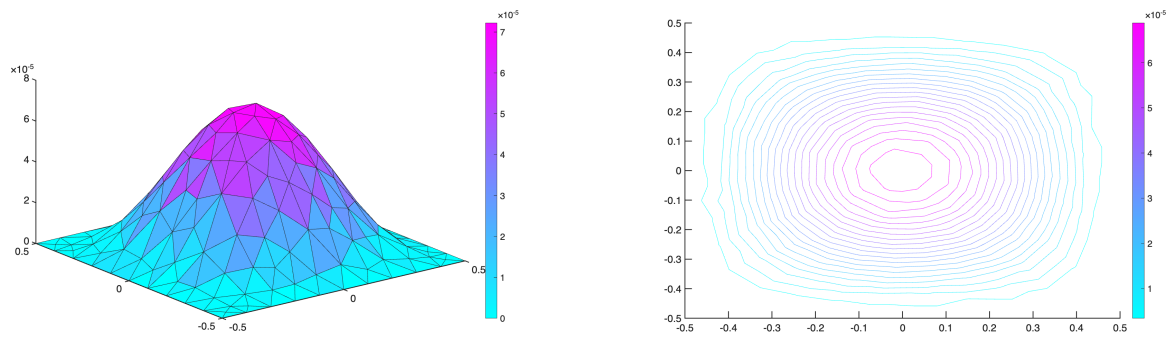


(b) Variance

Figure 7: Sobol convergence vs. N — P2.

Figure 8: Sobol convergence vs. N — P3.

Figures 9 and 10 show the full P3 mean and variance fields at $N = 200\,000$, which are nearly identical to the LHS results, confirming consistency between the two methods.

Figure 9: P3 mean field (Sobol, $N = 200\,000$): 3-D surface (left) and contour (right).Figure 10: P3 variance field (Sobol, $N = 200\,000$): 3-D surface (left) and contour (right).

Q3 — Sparse Clenshaw–Curtis Quadrature

Methodology

A Smolyak sparse grid of level ℓ combines one-dimensional Clenshaw–Curtis (CC) rules of varying orders such that the sum of the 1-D levels does not exceed $\ell + d - 1$. This avoids the exponential growth of the full

tensor product while retaining much of its accuracy for smooth functions.

Let $\{(y_n, w_n)\}_{n=1}^{N_\ell}$ be the resulting nodes and weights on $[-1, 1]^d$. The integral $\int_{[-1, 1]^d} f(\mathbf{y}) d\mathbf{y} \approx \sum_{n=1}^{N_\ell} w_n f(y_n)$, so the first two moments under a uniform distribution on $[-1, 1]^d$ are

$$\hat{\mu} = \frac{1}{2^d} \sum_{n=1}^{N_\ell} w_n f(y_n), \quad \hat{\sigma}^2 = \frac{1}{2^d} \sum_{n=1}^{N_\ell} w_n f(y_n)^2 - \hat{\mu}^2. \quad (1)$$

Mapping to general intervals. For parameters $\theta_i \sim \mathcal{U}[a_i, b_i]$ (as in P2), the affine map

$$\theta_i = a_i + \frac{b_i - a_i}{2}(y_i + 1), \quad y_i \in [-1, 1],$$

is applied to each node coordinate. The integration weights are scaled by $\prod_{i=1}^d \frac{b_i - a_i}{2}$, and the mean is recovered by dividing by the hyper-rectangle volume $\prod_{i=1}^d (b_i - a_i)$, giving

$$\hat{\mu} = \frac{\sum_{n=1}^{N_\ell} w_n \left(\prod_{i=1}^d \frac{b_i - a_i}{2} \right) f(\theta(y_n))}{\prod_{i=1}^d (b_i - a_i)}.$$

Code Structure

Q3.m retrieves CC nodes on $[-1, 1]^d$ and applies an affine map to rescale them to each problem's parameter domain before evaluation. The mean is computed as $(w \cdot f^\top) \times w_{\text{scale}}/V$, where $w_{\text{scale}} = \prod_i (b_i - a_i)/2$ corrects for the change of variables and $V = \prod_i (b_i - a_i)$ is the parameter volume; for P1 both cancel to $1/2^{10}$. P1 was capped at level 4 because the provided `sparse_grid_cc_weights` has an $\mathcal{O}(N^2)$ inner loop that stalls at level 5 ($N = 41\,265$).

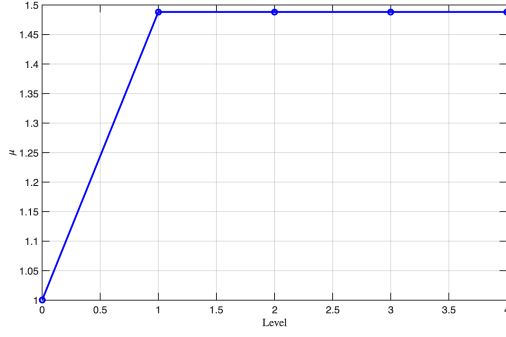
Results and Discussion

Unlike the sampling methods in Q1 and Q2, the sparse CC approach is deterministic: it sweeps over increasing quadrature levels and converges monotonically. Table 3 reports the estimated mean and variance at each level, along with the number of unique quadrature points.

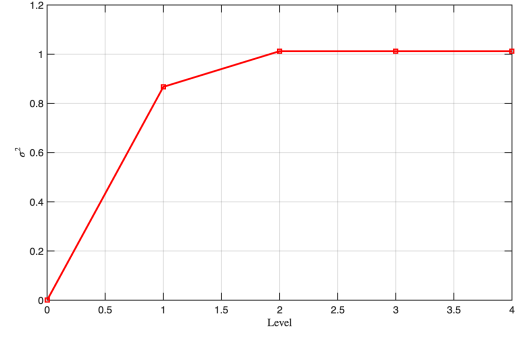
Table 3: Sparse CC results for P1 (levels 0–4), P2 (levels 0–6), and P3 (levels 0–6). P1 level 5 (41 265 pts) was omitted due to the $\mathcal{O}(N^2)$ cost of the provided weight routine at that grid size.

Lvl	P1			P2			P3		
	Pts	$\hat{\mu}$	$\hat{\sigma}^2$	Pts	$\hat{\mu}$	$\hat{\sigma}^2$	Pts	$\hat{\mu}$	$\hat{\sigma}^2$
0	1	1.0000	0	1	7.476×10^{-3}	0	1	6.240×10^{-2}	0
1	21	1.4882	0.8672	11	7.199×10^{-3}	6.212×10^{-4}	7	6.362×10^{-2}	7.593×10^{-5}
2	221	1.4882	1.0122	61	7.183×10^{-3}	6.494×10^{-4}	25	6.361×10^{-2}	7.227×10^{-5}
3	1 581	1.4882	1.0122	241	7.190×10^{-3}	6.497×10^{-4}	69	6.361×10^{-2}	7.221×10^{-5}
4	8 801	1.4882	1.0122	801	7.188×10^{-3}	6.495×10^{-4}	177	6.361×10^{-2}	7.221×10^{-5}
5	—	—	—	2 433	7.189×10^{-3}	6.496×10^{-4}	441	6.361×10^{-2}	7.221×10^{-5}
6	—	—	—	6 993	7.189×10^{-3}	6.496×10^{-4}	1 073	6.361×10^{-2}	7.221×10^{-5}

Figures 11–13 show convergence vs. quadrature level for all three problems.

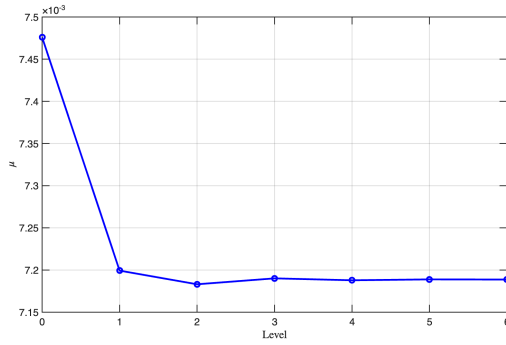


(a) Mean

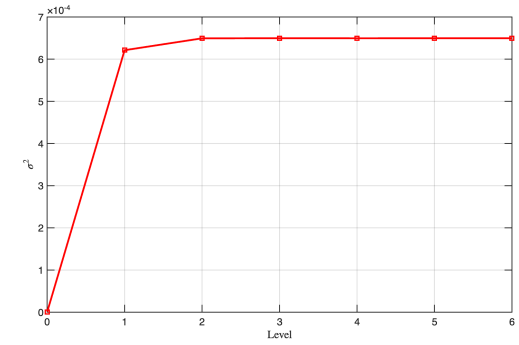


(b) Variance

Figure 11: Sparse CC convergence vs. level — P1.



(a) Mean



(b) Variance

Figure 12: Sparse CC convergence vs. level — P2.

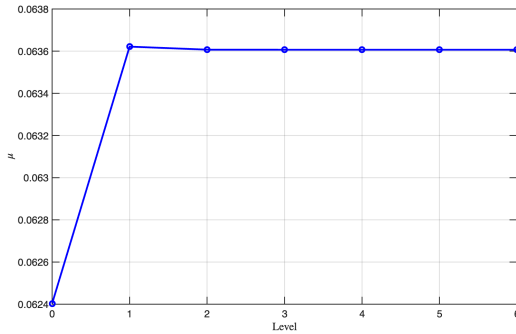
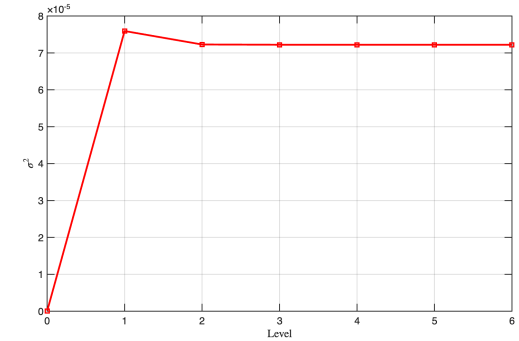
(a) Mean (u_1)(b) Variance (u_1)

Figure 13: Sparse CC convergence vs. level — P3.

The key observations are: for P1, both moments converge by level 2 (221 points), which is dramatically fewer than the $\mathcal{O}(10^4)$ samples needed by LHS/Sobol. For P2, the estimates stabilise by level 2–3 (61–241 points). For P3, convergence is achieved at level 3 with only 69 points. This rapid convergence stems from the nestedness of the CC rules: points from lower levels are reused at higher levels at no extra cost, and the Smolyak construction avoids the exponential growth of a full tensor product. The downside is the curse of

dimensionality: for $d = 10$ (P1) the grid already reaches 8 801 points at level 4, and the $\mathcal{O}(N^2)$ weight-assembly cost in the provided library prevented running level 5 (41 265 points).

Figures 14 and 15 show the P3 mean and variance fields computed at level 7, which are consistent with those from LHS and Sobol.

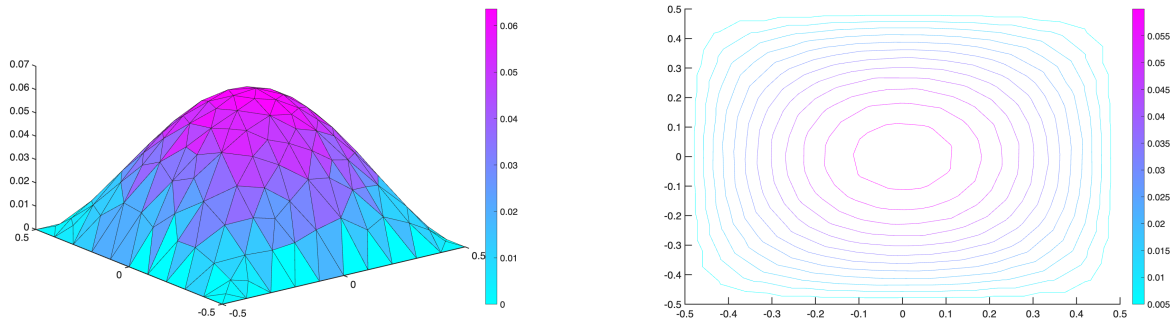


Figure 14: P3 mean field (Sparse CC, level 7): 3-D surface (left) and contour (right).

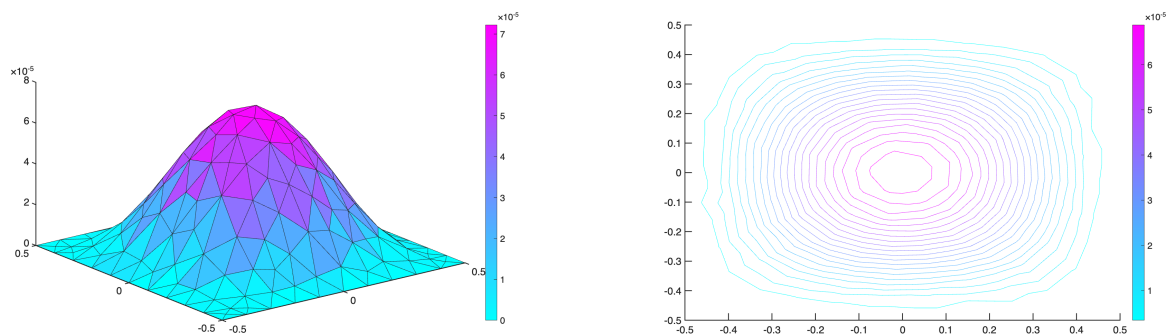


Figure 15: P3 variance field (Sparse CC, level 7): 3-D surface (left) and contour (right).

Comparison with Assignment 1 Monte Carlo

This section compares the converged estimates from LHS (Q1), Sobol QMC (Q2), and sparse Clenshaw–Curtis quadrature (Q3) against the classical Monte Carlo results obtained in Assignment 1. A direct comparison for P2 and P3 is given in Table 4, followed by a discussion of the key findings.

Note on P1. Assignment 1 used $x_i \sim \mathcal{N}(0, 1)$ for P1, whereas Assignment 2 requires uniform distributions ($x_i \sim \mathcal{U}[-1, 1]$). The resulting moments differ by construction and a direct comparison is therefore not meaningful; P1 is excluded from Table 4.

P2 and P3 — direct comparison. For P2 and P3, all four methods use identical parameter distributions, so results are directly comparable. The LHS, Sobol, and Sparse CC estimates agree with the Assignment 1 MC values to within 0.8% in the mean and 0.1% in the variance for both problems. This cross-validates all three Assignment 2 implementations and confirms that they converge to the correct statistical moments.

Efficiency comparison. The three methods differ substantially in the number of model evaluations required to achieve convergence:

Table 4: Converged moment estimates for P2 and P3 across all methods. P1 is excluded because the input distributions differ between assignments.

Method	P2		P3 (u_1)	
	$\hat{\mu}$	$\hat{\sigma}^2$	$\hat{\mu}$	$\hat{\sigma}^2$
MC A1 ($N = 200\,000$)	7.129×10^{-3}	6.490×10^{-4}	6.361×10^{-2}	7.209×10^{-5}
LHS ($N = 200\,000$)	7.186×10^{-3}	6.494×10^{-4}	6.361×10^{-2}	7.219×10^{-5}
Sobol ($N = 200\,000$)	7.189×10^{-3}	6.496×10^{-4}	6.361×10^{-2}	7.221×10^{-5}
Sparse CC (converged)	7.189×10^{-3}	6.496×10^{-4}	6.361×10^{-2}	7.221×10^{-5}

- **LHS** requires $N \approx 10\,000$ samples for stable estimates on all three problems. Its stratification reduces estimator variance relative to plain MC, but convergence still follows a stochastic $\mathcal{O}(N^{-1/2})$ rate.
- **Sobol QMC** converges monotonically and reaches the same accuracy with $N \approx 10\,000$ points. The deterministic low-discrepancy structure eliminates the random oscillations visible in LHS at small N .
- **Sparse CC** is by far the most efficient for smooth problems: P3 converges at level 3 with only **69 points**, P2 by level 3 with 241 points, and P1 by level 2 with 221 points. This represents a reduction of roughly two orders of magnitude relative to LHS/Sobol for the same accuracy.

Conclusions. All three Assignment 2 methods reproduce the Assignment 1 MC benchmarks for P2 and P3 to well within sampling uncertainty, confirming correctness. For smooth, low-to-moderate dimensional problems, Sparse CC is the clear winner in efficiency. LHS and Sobol are competitive with each other and preferable when the integrand is non-smooth or the dimension is too high for sparse grids to be practical.

Q4 — Importance Sampling with Unknown Normalisation

Problem Statement

We wish to evaluate

$$I = \int \phi(\mathbf{x}) p(\mathbf{x}) d\mathbf{x}, \quad p(\mathbf{x}) = \frac{q(\mathbf{x})}{Z},$$

where $q(\mathbf{x})$ is known pointwise but the normalisation constant Z is not. Since p is a valid probability density, integrating both sides of $p(\mathbf{x}) = q(\mathbf{x})/Z$ over its support and using $\int p(\mathbf{x}) d\mathbf{x} = 1$ gives

$$1 = \frac{1}{Z} \int q(\mathbf{x}) d\mathbf{x} \implies Z = \int q(\mathbf{x}) d\mathbf{x}.$$

Thus Z is the integral of q , which is itself unknown since we only know q pointwise.

Derivation

Step 1 — Introduce a proposal $g(\mathbf{x})$. Choose a density g that is easy to sample from and satisfies $g(\mathbf{x}) > 0$ wherever $q(\mathbf{x}) > 0$. Define the importance weight

$$w(\mathbf{x}) = \frac{q(\mathbf{x})}{g(\mathbf{x})}.$$

Step 2 — Rewrite I and Z as expectations under g .

$$Z = \int q(\mathbf{x}) d\mathbf{x} = \int \frac{q(\mathbf{x})}{g(\mathbf{x})} g(\mathbf{x}) d\mathbf{x} = \mathbb{E}_g[w(\mathbf{X})], \quad (2)$$

$$I = \frac{1}{Z} \int \phi(\mathbf{x}) q(\mathbf{x}) d\mathbf{x} = \frac{\mathbb{E}_g[\phi(\mathbf{X}) w(\mathbf{X})]}{\mathbb{E}_g[w(\mathbf{X})]}. \quad (3)$$

Equation (3) expresses I as a ratio of two g -expectations, eliminating the need to know Z explicitly.

Step 3 — Monte Carlo estimator. Draw $X_1, \dots, X_N \stackrel{\text{iid}}{\sim} g$. Form the empirical averages

$$\hat{Z} = \frac{1}{N} \sum_{i=1}^N w(X_i), \quad \hat{A} = \frac{1}{N} \sum_{i=1}^N \phi(X_i) w(X_i).$$

The *self-normalised importance sampling* (SNIS) estimator is

$$\hat{I} = \frac{\hat{A}}{\hat{Z}} = \frac{\sum_{i=1}^N \phi(X_i) w(X_i)}{\sum_{i=1}^N w(X_i)}. \quad (4)$$

Step 4 — Properties.

- **Consistency:** By the law of large numbers applied separately to numerator and denominator, $\hat{I} \rightarrow I$ as $N \rightarrow \infty$.
- **Bias:** $\hat{I}$ is biased for finite N (ratio of two random quantities), but the bias is $\mathcal{O}(1/N)$.
- **Variance:** Write $\hat{I} = h(\hat{A}, \hat{Z})$ where $h(a, b) = a/b$. The delta method linearises h around its limit (A_0, Z) where $A_0 = \mathbb{E}_g[\phi w] = IZ$:

$$\hat{I} - I \approx \frac{1}{Z}(\hat{A} - A_0) - \frac{I}{Z}(\hat{Z} - Z).$$

Substituting $\hat{A} = \frac{1}{N} \sum \phi_i w_i$ and $\hat{Z} = \frac{1}{N} \sum w_i$, this becomes

$$\hat{I} - I \approx \frac{1}{NZ} \sum_{i=1}^N [(\phi_i w_i - A_0) - I(w_i - Z)] = \frac{1}{NZ} \sum_{i=1}^N (\phi_i - I) w_i,$$

where the last step uses $A_0 - IZ = 0$. Since each term is i.i.d.,

$$\text{Var}(\hat{I}) \approx \frac{1}{NZ^2} \text{Var}_g[(\phi(\mathbf{X}) - I) w(\mathbf{X})],$$

so $\text{Var}(\hat{I})$ is proportional to $\text{Var}_g[(\phi - I) w]$. Choosing $g \propto |\phi| q$ makes $w = q/g \propto 1/|\phi|$, so $(\phi - I) w \propto (\phi - I)/|\phi|$. Under the optimal g , samples concentrate where $|\phi|$ is large, so the relative fluctuation I/ϕ stays close to 1, minimising the variance. In practice, g should have heavier tails than q to avoid a few large weights dominating the sum.

Concluding Remarks (Q4)

The SNIS estimator (4) handles the unknown normalisation Z by dividing two consistent sample averages. In practice, the choice of g is critical: if g has lighter tails than q , the weights $w(X_i)$ can become extremely large for rare samples, causing high estimator variance. Using a g with heavier tails (e.g. a Student- t when q is Gaussian-like) mitigates this degeneracy.

Q5 — Multilevel Monte Carlo

Key Insight

Consider a quantity of interest P defined as a functional of an SDE (or more generally any model with a discretisation parameter). Let $\ell = 0, 1, \dots, L$ index a hierarchy of discretisation levels, where level ℓ uses step-size $h_\ell = h_0 M^{-\ell}$ for some refinement factor $M \geq 2$. P_ℓ denotes the numerical approximation of P at level ℓ : coarse and cheap at $\ell = 0$, fine and expensive at $\ell = L$. The target accuracy is ε , meaning we want $\text{RMSE} \leq \varepsilon$.

Estimating $\mathbb{E}[P_L]$ directly with standard Monte Carlo costs $\mathcal{O}(\varepsilon^{-3})$ for a first-order solver: $N = \mathcal{O}(\varepsilon^{-2})$ samples are needed (from the CLT) and each costs $\mathcal{O}(h_L^{-1}) = \mathcal{O}(\varepsilon^{-1})$ because $h_L = \mathcal{O}(\varepsilon)$ is required to keep the bias below ε . MLMC avoids this by rewriting the expectation as a telescoping sum,

$$\mathbb{E}[P_L] = \mathbb{E}[P_0] + \sum_{\ell=1}^L \mathbb{E}[P_\ell - P_{\ell-1}],$$

and estimating each correction independently. The key is that P_ℓ and $P_{\ell-1}$ are evaluated on the *same* random realisation, so their difference has much smaller variance than either term alone. Two exponents govern the complexity:

- β — rate of variance decay: $\text{Var}[P_\ell - P_{\ell-1}] = \mathcal{O}(h_\ell^\beta)$. A larger β means corrections become small quickly, so coarse levels dominate.
- γ — cost per sample: $C_\ell = \mathcal{O}(h_\ell^{-\gamma})$. For a 1-D ODE solver $\gamma = 1$; for a 2-D PDE solver $\gamma = 2$.

Allocating $N_\ell \propto \sqrt{V_\ell/C_\ell}$ optimally across levels, Giles proves the total cost to achieve RMSE ε depends on the relative size of β and γ , as summarised in Table 5.

Table 5: MLMC complexity vs. standard MC, as a function of β and γ . α is the weak convergence order: $|\mathbb{E}[P - P_\ell]| = \mathcal{O}(h_\ell^\alpha)$.

Condition	Interpretation	MLMC cost	Standard MC cost
$\beta > \gamma$	Variance decays fast; coarse levels dominate	$\mathcal{O}(\varepsilon^{-2})$	
$\beta = \gamma$	Cost evenly spread across levels	$\mathcal{O}(\varepsilon^{-2}(\log \varepsilon)^2)$	$\mathcal{O}(\varepsilon^{-2-\gamma/\alpha})$
$\beta < \gamma$	Fine levels expensive; limited gain	$\mathcal{O}(\varepsilon^{-2-(\gamma-\beta)/\alpha})$	

For a first-order ODE solver ($\alpha = \beta = \gamma = 1$), MLMC achieves $\mathcal{O}(\varepsilon^{-2} \log^2 \varepsilon)$, a near-optimal improvement over the $\mathcal{O}(\varepsilon^{-3})$ standard MC cost.

Relation to the Control Variate Method

With two levels ($L = 1$), the MLMC estimator is

$$\hat{P}^{\text{ML}} = \underbrace{\frac{1}{N_0} \sum_{i=1}^{N_0} P_0^{(i)}}_{\text{coarse MC estimate}} + \underbrace{\frac{1}{N_1} \sum_{i=1}^{N_1} (P_1^{(i)} - P_0^{(i)})}_{\text{correction}}.$$

This closely resembles the control variate (CV) estimator from class,

$$\hat{I}^{\text{CV}} = \frac{1}{N} \sum_{i=1}^N [\phi(X^{(i)}) - \lambda g(X^{(i)})] + \lambda \mu_g,$$

where g is a correlated surrogate with known mean μ_g and optimal coefficient $\lambda^* = \text{Cov}(\phi, g)/\text{Var}(g)$. Table 6 summarises the similarities and differences.

Table 6: Two-level MLMC vs. control variates.

	Control Variates	Two-level MLMC
Similarities		
Variance reduction	Subtract correlated $g(X)$ to reduce variance of integrand.	Subtract correlated P_0 to reduce variance of correction.
Estimator form	$\hat{I}^{\text{CV}} = \text{MC estimate of } (\phi - \lambda g) + \lambda \mu_g$.	$\hat{P}^{\text{ML}} = \text{MC estimate of } P_0 + \text{MC estimate of } (P_1 - P_0)$.
Unbiasedness	Both produce unbiased estimators.	
Differences		
Mean of control	$\mu_g = \mathbb{E}[g(X)]$ must be known <i>analytically</i> .	$\mathbb{E}[P_0]$ is estimated numerically — no closed form needed.
Coupling	ϕ and g are evaluated independently or by design choice.	P_1 and P_0 share the <i>same</i> random realisation by construction.
Coefficient	λ^* is tuned to minimise variance.	No free coefficient; variance reduction comes purely from coupling.
Scalability	Multiple controls require solving a system for λ_i ; N is fixed.	Extends naturally to $L > 2$ levels with optimal N_ℓ allocation.

Application to P1–P3

For **P2** (mass–spring–damper), MLMC applies directly: define a hierarchy of time-steps $h_\ell = T \cdot M^{-\ell}$ and, for each parameter draw (m, c, k, F_0, ω) , solve the ODE at two successive resolutions to form the correction $P_\ell - P_{\ell-1}$. With first-order convergence ($\alpha = \beta = \gamma = 1$) the cost drops from $\mathcal{O}(\varepsilon^{-3})$ to $\mathcal{O}(\varepsilon^{-2} \log^2 \varepsilon)$.

For **P3** (stochastic diffusion), the hierarchy is defined by mesh refinement: successive grids double the resolution, and for each θ draw the coarse and fine linear systems are assembled and solved with the same parameters. For a second-order elliptic problem on a 2-D mesh, $\beta \approx 2$ and $\gamma = 2$, so $\beta = \gamma$ and the cost is $\mathcal{O}(\varepsilon^{-2} \log^2 \varepsilon)$.

For **P1** (quadratic), there is no discretisation parameter, so the classical MLMC framework does not apply directly. A dimension-incremental hierarchy could be constructed by truncating to $d_\ell = 2^\ell$ dimensions at each level, with missing components set to zero (their mean under $\mathcal{U}[-1, 1]$). The correction variance decays because the higher-index coefficients $1/\sqrt{i}$ decrease, but the gain over direct sampling would be modest given how cheaply P1 evaluates.

Q6 — O’Hagan’s Critique of Monte Carlo

(a) Central Critique

O’Hagan frames the problem around the basic Monte Carlo integral $k = \int f(x) dx$. The integral is rewritten as $k = \mathbb{E}_g[r(X)]$ where $r(x) = f(x)/g(x)$ and g is an arbitrary sampling density, so that the sample mean $\bar{r} = n^{-1} \sum r(x_i)$ serves as an unbiased estimator. He then raises two specific objections.

First objection: dependence on the arbitrary sampling density g . O’Hagan points out that the estimator $\bar{r}$ depends on the choice of g . If two analysts observe exactly the same sample points $x_1, \dots, x_n$ and compute exactly the same function values $f(x_1), \dots, f(x_n)$, but one used sampling density g while the other used a different density g_1 , they will produce different estimates $\frac{1}{n} \sum f(x_i)/g(x_i) \neq \frac{1}{n} \sum f(x_i)/g_1(x_i)$. The choice of g is arbitrary and carries no information about f or its integral k . Two analysts with identical data about f should not arrive at different answers simply because they used different sampling schemes. O’Hagan notes that this violates the Likelihood Principle: all information about k is contained in the observed function values, not in the mechanism used to generate the sample locations.

Second objection: ignoring the sample locations x_i . The Monte Carlo estimator $\bar{r}$ uses only the function values $f(x_i)$ and ignores where they were evaluated. O’Hagan illustrates this with a simple example: suppose $n = 3$ and it happens that $x_2 = x_3$. Then $f(x_2) = f(x_3)$, and computing $f(x_3)$ provides zero

additional information about k . Yet the Monte Carlo estimator still gives equal weight to all three observations. A sensible estimator would recognize that only two distinct pieces of information are available. The same argument applies when x_2 and x_3 are not exactly equal but very close: $f(x_3)$ provides very little new information once $f(x_2)$ is known, because a smooth function takes similar values at nearby points. Giving both observations equal weight is unreasonable.

This second objection extends to the variance estimate as well. When sample points cluster together, the internal Monte Carlo variance estimate S_r^2/n becomes optimistic in two ways: the sample variance S_r^2 is deflated because nearby function values are similar, and dividing by n overstates the effective number of independent observations. O'Hagan works through the $n = 3$, $x_2 = x_3$ case explicitly: the Monte Carlo treatment gives an estimated variance of $\{r(x_1) - r(x_2)\}^2/9$, whereas treating this correctly as $n = 2$ gives $\{r(x_1) - r(x_2)\}^2/4$, which is more than twice as large. So the Monte Carlo procedure not only produces a worse estimate of k , it also reports a misleadingly low uncertainty. Conversely, when the x_i happen to be well spread out, the estimate of k is more accurate than the Monte Carlo variance formula suggests, making the reported uncertainty unnecessarily pessimistic.

The philosophical inconsistency that ties both objections together is that Monte Carlo assumes the integrand has enough regularity for the integral to exist and be finite, but then discards that regularity when constructing the estimate. A smooth function carries information between sample points: knowing $f(x)$ constrains what $f(x')$ can be for nearby x' . Monte Carlo ignores this constraint entirely, treating each evaluation as statistically independent of every other regardless of spatial proximity. Any method that accounts for the locations of the sample points and the smoothness of f should be able to extract more information per function evaluation.

(b) Course Methods Most Aligned with Addressing O'Hagan's Concerns

Among the methods studied in this course, sparse grid quadrature is the most aligned with addressing O'Hagan's concerns. It directly resolves both of his objections.

First, there is no arbitrary sampling density g . The quadrature nodes and weights are fixed by the rule and the dimension of the problem, so two analysts applying the same Clenshaw-Curtis sparse grid will always produce the same estimate. The answer depends only on the function values, not on any auxiliary choice made by the analyst.

Second, sparse grids fully exploit the locations of the evaluation points and the smoothness of the integrand. The Smolyak construction places nodes so that polynomials up to a target total degree are integrated exactly, and the weights are chosen to be optimal for that specific node set. Nearby nodes do not receive redundant equal weight the way Monte Carlo treats them; instead, the entire node-weight structure is designed so that every evaluation contributes maximally to the accuracy of the integral. The convergence rate for a function in C^r is $O(N_Q^{-r}(\log N_Q)^{(r+1)(d-1)})$, which improves directly with the smoothness r . This is the core of O'Hagan's argument: if the integrand is smooth, a method that accounts for that smoothness should converge faster than the $O(N^{-1/2})$ rate of Monte Carlo, which is independent of smoothness.

The results from this assignment illustrate the point concretely. As shown in Table 4 and the Q3 discussion, the sparse Clenshaw-Curtis rule converged to the correct mean and variance of P3 with only 69 quadrature points (level 3), and of P2 with 241 points (level 3). LHS and Sobol QMC required on the order of 10 000 samples to reach the same accuracy on the same problems. For P1, the sparse grid converged at level 2 with 221 points, while LHS and Sobol needed roughly 10 000 to 20 000 samples. This difference of one to two orders of magnitude in the number of function evaluations is exactly the kind of efficiency gain that O'Hagan argues should be achievable by any method that properly uses the regularity of the integrand, rather than discarding it as Monte Carlo does.

References

- [1] M. B. Giles. Multilevel Monte Carlo path simulation. *Operations Research*, 56(3):607–617, 2008.
- [2] A. O'Hagan. Monte Carlo is fundamentally unsound. *Journal of the Royal Statistical Society, Series D (The Statistician)*, 36(2/3):247–249, 1987.